Guía de ejercicios

Objetos en JavaScript

Sesión 2 · Fundamentos de programación

Con los arrays aprendiste a guardar muchos valores en fila y a llegar a cada uno por su posición. Pero hay datos que no son una fila: un cliente tiene nombre, teléfono y barrio, y no tiene sentido preguntar cuál es "el dato número 2" de un cliente. Para eso existen los objetos: guardan valores con nombre.

La guía tiene cuatro bloques. Los tres primeros se resuelven con acceso por punto, corchetes y el ciclo `for...in`. El cuarto introduce `Object.keys`, `Object.values` y `Object.entries`, y te pide comparar con lo que ya construiste a mano.

Reglas de la guía

- Puedes usar `for` y `while`. Desde el bloque 2 se habilita `for...in`. Nada de `do...while`.
- Los datos se leen con `prompt` y se muestran con `console.log`. Puedes usar `console.log(objeto)` para revisar mientras programas, pero la salida final debe ser texto formateado como en los ejemplos.
- Hay tres tipos de funciones. Las de **lectura** (`leer...`) pueden usar `prompt`. Las de **mostrar** (`mostrar...`) pueden usar `console.log`. Las de **cálculo** reciben datos por parámetro y retornan un resultado: no usan `prompt` ni `console.log`.
- Nombra las funciones con un verbo en camelCase y los objetos en singular (`cliente`, `producto`, `reserva`). Los arrays siguen en plural.
- **Bloques 1 a 3:** están prohibidos `Object.keys`, `Object.values`, `Object.entries`, `Object.assign`, el operador `...` (spread), `JSON.stringify`, `JSON.parse` y `structuredClone`. Sí puedes usar todo lo de la guía de arrays, incluidos `forEach`, `map`, `filter` y `find`, además de `split`, `toLowerCase`, `Math.round` y `toFixed`.
- **Bloque 4:** se habilitan `Object.keys`, `Object.values` y `Object.entries`.
- Nada de arrays de objetos todavía. Un objeto puede tener un array adentro, pero no al revés. Eso es la próxima sesión.
- Varios ejercicios reutilizan funciones de esta guía o de la de arrays (por ejemplo `unirConComas`). Cópialas; no las reescribas desde cero.
- **Antes de programar cada ejercicio, haz la prueba de escritorio con los datos del ejemplo.**

En los ejemplos de ejecución, las líneas que empiezan con `[prompt]` muestran lo que el usuario escribe. Las demás son lo que imprime `console.log`.

¿Qué es un objeto?

Piensa en una cédula: no es una lista de datos sueltos, es un documento con **campos con nombre**. Nombres, apellidos, número, fecha de nacimiento. Para encontrar el número no cuentas posiciones: buscas el campo "número". Un objeto funciona igual: cada valor se guarda con un nombre, llamado **clave** o **propiedad**.

```
let cliente = {
  nombre: "Jorge Díaz",
  telefono: "3001234567",
  barrio: "La Playa",
  pedidos: 7
};
```

Cada línea es un par **clave: valor**, separados por comas. Las claves son textos; los valores pueden ser de cualquier tipo, incluso arrays u otros objetos.

¿Array u objeto?

	Array	Objeto
Cómo llegas a un dato	Por posición: `precios[2]`	Por nombre: `cliente.barrio`

	Array	Objeto
Para qué sirve	Una lista de cosas del mismo tipo	Una sola cosa con varias características
Ejemplo	Los precios de 30 productos	Los datos de un producto
¿El orden importa?	Sí	No lo uses para ordenar
¿Cuántos datos hay?	`length`	Lo descubres en el bloque 2

Pregunta útil para decidir: si tiene sentido decir "el tercero", es un array. Si tiene sentido decir "el teléfono", es un objeto.

Leer: punto y corchetes

```
console.log(cliente.nombre);      // Jorge Díaz
console.log(cliente["barrio"]);   // La Playa

let campo = "telefono";
console.log(cliente[campo]);      // 3001234567
console.log(cliente.campo);       // undefined: busca una clave llamada "campo"
```

Usa el **punto** cuando sabes el nombre de la clave al escribir el código. Usa **corchetes** cuando el nombre de la clave está guardado en una variable, porque lo escribió el usuario o viene de un ciclo.

Agregar, modificar y eliminar

```
cliente.barrio = "Pablo VI";     // modifica: la clave ya existía
cliente.correo = "jd@mail.com";  // agrega: la clave no existía
delete cliente.pedidos;           // elimina la clave y su valor
```

Asignar a una clave que no existe no da error: la crea. Eso es cómodo y también es la fuente del error más común con objetos, como vas a ver en la prueba de escritorio.

¿Existe la clave?

```
console.log("correo" in cliente);   // true
console.log("pedidos" in cliente);  // false, la eliminamos
console.log(cliente.pedidos);       // undefined
```

Igual que con los arrays, leer una clave que no existe da `undefined` sin error. Pero hay una diferencia importante: si intentas leer una propiedad **de** algo que es `undefined`, el programa sí se rompe.

```
console.log(cliente.direccion);         // undefined, sin error
console.log(cliente.direccion.ciudad);  // TypeError: el programa se detiene
```

Los objetos se pasan por referencia

Lo que viste con arrays aplica igual aquí: si le pasas un objeto a una función y la función lo modifica, el original cambia. Y si escribes `let copia = cliente;` no creas otro objeto, creas otro nombre para el mismo. Por eso `===` entre dos objetos solo da `true` si son literalmente el mismo objeto, aunque tengan los mismos datos.

Prueba de escritorio con objetos

Con objetos la tabla necesita dos columnas nuevas: el **estado completo del objeto** después de cada paso, y el valor que tenía la clave **antes** de modificarla. Esa segunda columna es la que atrapa los errores.

Ejemplo paso a paso

Problema: una caja registradora acumula lo recibido por método de pago. Se leen 4 pagos, cada uno con su método (`efectivo` o `tarjeta`) y su monto.

```
function registrarPago(caja, metodo, monto) {
  caja[metodo] = caja[metodo] + monto;
}

let caja = { efectivo: 0, tarjeta: 0 };
for (let i = 0; i < 4; i++) {
  let metodo = prompt("Método del pago " + (i + 1) + ":");
  let monto = Number(prompt("Monto:"));
  registrarPago(caja, metodo, monto);
}
console.log("Efectivo: " + caja.efectivo);
console.log("Tarjeta: " + caja.tarjeta);
```

Paso 1: elige los datos y calcula el resultado esperado

Pagos: efectivo 15000, tarjeta 40000, efectivo 8000, tarjeta 12000. Resultado esperado: **efectivo 23000 y tarjeta 52000**.

Paso 2: arma la tabla

Paso	i	metodo	monto	caja[metodo] antes	caja
1	0	efectivo	15000	0	{ efectivo: 15000, tarjeta: 0 }
2	1	tarjeta	40000	0	{ efectivo: 15000, tarjeta: 40000 }
3	2	efectivo	8000	15000	{ efectivo: 23000, tarjeta: 40000 }
4	3	tarjeta	12000	40000	{ efectivo: 23000, tarjeta: 52000 }
5	4	—	—	—	Sale del ciclo (4 < 4 es false)

Paso 3: compara

Esperabas 23000 y 52000, y la tabla da lo mismo. La lógica está bien.

El error que la prueba atrapa

Ahora repite la tabla suponiendo que en el pago 3 el cajero escribió `Efectivo`, con mayúscula:

Paso	i	metodo	monto	caja[metodo] antes	caja
3	2	Efectivo	8000	undefined	{ efectivo: 15000, tarjeta: 40000, Efectivo: NaN }

`caja["Efectivo"]` no existe, así que vale `undefined`. `undefined + 8000` da `NaN`, y como asignar a una clave inexistente la crea, el objeto ganó una clave nueva que nadie planeó. El programa no se rompe e imprime efectivo 15000: un resultado equivocado sin ningún aviso.

Regla práctica: si en la columna "antes" aparece `undefined` cuando esperabas un número, estás creando una clave por accidente. Valida que la clave exista antes de modificarla, o normaliza el texto con `toLowerCase`.

Bloque 1. Crear, leer y modificar

Ejercicio 1. Ficha del aprendiz

Qué practica: Crear un objeto con datos leídos, acceso por punto y propiedades calculadas.

Enunciado:

Leer el nombre, la edad, el número de ficha y el programa de un aprendiz, y guardarlos en un objeto. Agregar al objeto una propiedad `mayorDeEdad` con `true` o `false` según la edad. Mostrar la ficha.

Ejemplo de ejecución:

```
[prompt] Nombre: Valentina Rojas
[prompt] Edad: 19
[prompt] Número de ficha: 2847561
[prompt] Programa: Análisis y Desarrollo de Software
--- Ficha del aprendiz ---
Nombre: Valentina Rojas
Edad: 19 años
Ficha: 2847561
Programa: Análisis y Desarrollo de Software
Mayor de edad: sí
```

Funciones sugeridas:

- `leerAprendiz()`: pide los datos y retorna el objeto completo.
- `mostrarFicha(aprendiz)`: imprime la ficha.

Pista: ¿Guardaste la edad como texto o como número? Compruébalo con `typeof aprendiz.edad`. ¿Por qué conviene guardar `mayorDeEdad` como `true`/`false` y convertirlo a "sí"/"no" solo al mostrarlo?

Ejercicio 2. Actualizar el perfil

Qué practica: Corchetes con una clave guardada en variable y comprobar si una clave existe.

Enunciado:

El programa arranca con este objeto:

```
let cliente = { nombre: "Jorge Díaz", telefono: "3001234567", barrio: "La Playa",
ciudad: "San Gil" };
```

El usuario escribe qué dato quiere cambiar y el valor nuevo. Si la clave no existe, se avisa y no se crea. Se repite hasta que escriba "fin". Al final se muestra el perfil.

Ejemplo de ejecución:

```
[prompt] ¿Qué dato quieres cambiar? barrio
[prompt] Nuevo valor: Pablo VI
Actualizado: barrio = Pablo VI
[prompt] ¿Qué dato quieres cambiar? correo
El dato correo no existe
[prompt] ¿Qué dato quieres cambiar? fin
Perfil final:
Nombre: Jorge Díaz
Teléfono: 3001234567
```

```
Barrio: Pablo VI
Ciudad: San Gil
```

Funciones sugeridas:

- `existeClave(objeto, clave)`: retorna `true` o `false`.
- `actualizarDato(objeto, clave, valor)`: modifica el objeto.
- `mostrarCliente(cliente)`: imprime el perfil.

Pista: Escribe en un comentario qué busca `cliente.clave` y qué busca `cliente[clave]`. ¿Por qué solo una de las dos sirve aquí?

Ejercicio 3. Producto en oferta

Qué practica: Construir un objeto nuevo a partir de otro sin modificar el original.

Enunciado:

El programa arranca con este objeto:

```
let producto = { nombre: "Morral", precio: 85000, stock: 12 };
```

Leer un porcentaje de descuento. Una función debe retornar un objeto **nuevo** con el precio rebajado y una propiedad extra `precioAnterior`. Las demás propiedades se copian igual. Mostrar ambos objetos para demostrar que el original quedó intacto.

Ejemplo de ejecución:

```
[prompt] Porcentaje de descuento: 20
Original: Morral - $85000 - 12 unidades
En oferta: Morral - $68000 (antes $85000) - 12 unidades
```

Funciones sugeridas:

- `aplicarDescuento(producto, porcentaje)`: retorna el objeto nuevo.
- `mostrarProducto(producto)`: imprime una línea; si tiene `precioAnterior`, lo muestra entre paréntesis.

Pista: Antes de hacerlo bien, haz la prueba mal: dentro de la función escribe `let oferta = producto;` y cambia `oferta.precio`. ¿Qué imprime el original? ¿Por qué pasó?

Ejercicio 4. Reserva de rafting

Qué practica: Agregar y eliminar propiedades según decisiones del usuario.

Enunciado:

Leer el nombre del cliente y la cantidad de adultos, y crear una reserva con la actividad "Rafting río Fonce" y `seguro: true`. Preguntar si van menores; si sí, agregar la propiedad `menores` con su cantidad. Preguntar si rechaza el seguro; si sí, eliminar la propiedad `seguro`. Calcular el total: $70000 por adulto, $50000 por menor y $8000 por persona si hay seguro. Mostrar solo los datos que existen en la reserva.

Ejemplo de ejecución:

```
[prompt] Cliente: Camila Peña
[prompt] Adultos: 2
```

```
[prompt] ¿Van menores de edad? (si/no): si
[prompt] ¿Cuántos menores?: 1
[prompt] ¿Rechaza el seguro? (si/no): si
--- Reserva ---
Cliente: Camila Peña
Actividad: Rafting río Fonce
Adultos: 2
Menores: 1
Seguro: no incluido
Total: $190000
```

Funciones sugeridas:

- `crearReserva(cliente, adultos)`: retorna el objeto inicial.
- `calcularTotal(reserva)`: retorna el total según las propiedades que existan.
- `mostrarReserva(reserva)`: imprime la reserva.

Pista: Después de `delete reserva.seguro`, ¿qué vale `reserva.seguro`? ¿Qué diferencia hay con haber escrito `reserva.seguro = false`? Y si no van menores, ¿qué vale `reserva.menores` dentro de `calcularTotal`?

Bloque 2. Recorrer y usar objetos como diccionarios

Hasta ahora sabías el nombre de cada clave. Pero a veces no lo sabes: las claves las escribe el usuario o el objeto crece mientras el programa corre. Para visitar todas las claves de un objeto existe `for...in`:

```
let notas = { matematicas: 4.2, ingles: 3.1 };

for (let materia in notas) {
  console.log(materia + ": " + notas[materia]);
}
// matematicas: 4.2
// ingles: 3.1
```

En cada vuelta, la variable (`materia`) toma el **nombre** de una clave, no su valor. Para el valor usas corchetes: `notas[materia]`. Fíjate que aquí el punto no sirve, por la misma razón del ejercicio 2.

Ejercicio 5. Boletín de notas

Qué practica: `for...in`, clave y valor, y contar claves sin `length`.

Enunciado:

El programa arranca con este objeto:

```
let notas = { matematicas: 4.2, ingles: 3.1, programacion: 4.8, etica: 3.9 };
```

Mostrar cada materia con su nota, el promedio con un decimal, la materia con mejor nota y las materias por reforzar (nota menor a 3.5). Las funciones deben seguir funcionando si agregas una quinta materia sin tocarlas.

Ejemplo de ejecución:

```
matematicas: 4.2
ingles: 3.1
programacion: 4.8
etica: 3.9
Promedio: 4.0
Mejor materia: programacion
Por reforzar: ingles
```

Funciones sugeridas:

- `contarClaves(objeto)`: retorna cuántas claves tiene. La vas a reutilizar.
- `calcularPromedioObjeto(objeto)`: retorna el promedio de los valores.
- `buscarClaveMayor(objeto)`: retorna la **clave** del valor más alto. También se reutiliza.
- `filtrarClavesMenores(objeto, limite)`: retorna un array con las claves cuyo valor es menor que `limite`.

Pista: Con un array sabías cuántos elementos había con `length`. ¿Qué hace `notas.length`? Pruébalo. ¿Cómo cuentas las claves con lo que tienes?

Ejercicio 6. Glosario de backend

Qué practica: Un objeto como diccionario: buscar por clave sin recorrer, y agregar claves nuevas.

Enunciado:

El programa arranca con este glosario:

```
let glosario = {
  api: "Conjunto de reglas para que dos programas se comuniquen.",
  servidor: "Programa que recibe peticiones y devuelve respuestas.",
  ruta: "Dirección a la que llega una petición, como /productos.",
  middleware: "Función que se ejecuta entre la petición y la respuesta.",
  endpoint: "Combinación de un método HTTP y una ruta."
};
```

El usuario escribe términos hasta "fin" y el programa muestra la definición, sin importar mayúsculas. Si el término no está, pregunta si quiere agregarlo y, si sí, pide la definición y lo agrega. Al final dice cuántos términos tiene el glosario.

Ejemplo de ejecución:

```
[prompt] Término: API
api: Conjunto de reglas para que dos programas se comuniquen.
[prompt] Término: cookie
"cookie" no está en el glosario
[prompt] ¿Quieres agregarlo? (si/no): si
[prompt] Definición: Dato pequeño que el servidor guarda en el navegador.
Agregado
[prompt] Término: fin
El glosario tiene 6 términos
```

Funciones sugeridas:

- `buscarDefinicion(glosario, termino)`: retorna la definición o `undefined`.
- `contarClaves(objeto)`: la del ejercicio 5.

Pista: En la guía de arrays, `existeEnLista` necesitaba un ciclo para buscar. ¿Por qué `buscarDefinicion` no lo necesita? Responde en un comentario.

Ejercicio 7. Contador de palabras

Qué practica: Un objeto como contador cuyas claves aparecen mientras el programa corre.

Enunciado:

Leer una frase, pasarla a minúsculas y separarla en palabras con `split(" ")`. Contar cuántas veces aparece cada palabra usando un objeto. Mostrar cada palabra con su conteo en el orden en que apareció por primera vez, cuántas palabras distintas hay y cuál es la más repetida. Si hay empate, gana la que apareció primero.

Ejemplo de ejecución:

```
[prompt] Frase: el que madruga dios lo ayuda y el que no madruga también
el: 2
que: 2
madruga: 2
dios: 1
```

```
lo: 1
ayuda: 1
y: 1
no: 1
también: 1
Palabras distintas: 9
Más repetida: el (2 veces)
```

Funciones sugeridas:

- `contarPalabras(frase)`: retorna el objeto contador.
- `contarClaves` y `buscarClaveMayor`: las del ejercicio 5.

Pista: En la guía de arrays usaste un array de 5 contadores porque las estrellas iban de 1 a 5. ¿Por qué aquí no sirve? Cuando una palabra llega por primera vez, ¿qué vale `contador[palabra]`? Revisa la prueba de escritorio de la caja: es el mismo error.

Ejercicio 8. Inventario del local

Qué practica: Modificar valores con una clave dinámica, validando en el orden correcto.

Enunciado:

El programa arranca con este inventario:

```
let inventario = { arepas: 20, empanadas: 35, jugos: 15, tintos: 40 };
```

Se registran ventas: producto y cantidad, hasta que el producto sea "fin". Si el producto no existe o no hay suficiente, se avisa y no se modifica nada. Al final se muestra el inventario y los productos agotados.

Ejemplo de ejecución:

```
[prompt] Producto: empanadas
[prompt] Cantidad: 10
Vendidas 10 empanadas. Quedan 25
[prompt] Producto: jugos
[prompt] Cantidad: 15
Vendidas 15 jugos. Quedan 0
[prompt] Producto: pandebono
No vendemos pandebono
[prompt] Producto: arepas
[prompt] Cantidad: 30
Solo quedan 20 arepas
[prompt] Producto: fin
Inventario final:
arepas: 20
empanadas: 25
jugos: 0
tintos: 40
Agotados: jugos
```

Funciones sugeridas:

- `vender(inventario, producto, cantidad)`: modifica el inventario si se puede y retorna el mensaje que se debe mostrar (no lo imprime).
- `obtenerAgotados(inventario)`: retorna un array con las claves en 0.
- `unirConComas(lista)`: la de la guía de arrays.

Pista: Si validas primero el stock y después la existencia, ¿qué pasa con "pandebono"? Haz la prueba de escritorio: ¿cuánto vale `inventario["pandebono"] < 30`?

Bloque 3. Objetos con objetos y arrays adentro

Un valor de un objeto puede ser otro objeto o un array. Para llegar adentro encadenas accesos: `cliente.direccion.barrio` o `cliente.telefonos[0]`. Cada punto o corchete baja un nivel.

Ejercicio 9. Ficha con dirección

Qué practica: Acceso encadenado y protegerse cuando un nivel no existe.

Enunciado:

El programa arranca con estos dos objetos:

```
let cliente = {
  nombre: "Laura Gómez",
  telefonos: ["3104567890", "6077241234"],
  direccion: { calle: "Calle 12 # 9-45", barrio: "Centro", ciudad: "San Gil" }
};
let clienteNuevo = { nombre: "Pedro Ruiz", telefonos: [] };
```

Mostrar la ficha de `cliente`, leer un teléfono nuevo, un barrio y una calle nuevos, actualizar el objeto y volver a mostrar la ficha. Al final, mostrar la ciudad de `clienteNuevo` usando la misma función que con `cliente`, sin que el programa se rompa.

Ejemplo de ejecución:

```
--- Antes ---
Laura Gómez
Teléfonos: 3104567890, 6077241234
Dirección: Calle 12 # 9-45, Centro, San Gil
[prompt] Nuevo teléfono: 3158889900
[prompt] Nuevo barrio: Villa Olímpica
[prompt] Nueva calle: Carrera 5 # 20-10
--- Después ---
Laura Gómez
Teléfonos: 3104567890, 6077241234, 3158889900
Dirección: Carrera 5 # 20-10, Villa Olímpica, San Gil
Ciudad de Pedro Ruiz: sin dirección
```

Funciones sugeridas:

- `mostrarFicha(cliente)`: imprime la ficha.
- `cambiarDireccion(cliente, calle, barrio)`: modifica la dirección.
- `obtenerCiudad(cliente)`: retorna la ciudad o el texto "sin dirección".

Pista: Escribe `console.log(clienteNuevo.direccion.ciudad)` y ejecútalo. A diferencia de los arrays, aquí el programa sí se detiene. ¿Por qué `clienteNuevo.direccion` da `undefined` sin problema y el siguiente paso explota? ¿Qué revisas antes de bajar un nivel?

Ejercicio 10. El clon que no era clon

Qué practica: Referencias: distinguir cuándo dos variables apuntan al mismo objeto y copiar objetos anidados.

Enunciado:

Parte A. Copia este código y, **antes de ejecutarlo**, escribe en un comentario lo que crees que imprime la última línea y por qué. Después ejecútalo. Si te equivocaste, explica en otro comentario qué fue lo que no viste. Equivocarse aquí no resta: dejarlo sin explicar sí.

```
let pedido = { cliente: "Ana", total: 30000, entrega: { barrio: "Centro", hora: "7:00
pm" } };

let copia1 = pedido;
copia1.total = 45000;

let copia2 = { cliente: pedido.cliente, total: pedido.total, entrega: pedido.entrega
};
copia2.cliente = "Luis";
copia2.entrega.barrio = "La Playa";

console.log(pedido.total, pedido.cliente, pedido.entrega.barrio);
```

Parte B. Escribe una función que haga una copia completa del pedido, de modo que cambiar el barrio de la copia no afecte el original. Pruébala.

Ejemplo de ejecución:

```
(Salida de la parte B, con el pedido original de la parte A antes de modificarlo)
Barrio del original: Centro
Barrio de la copia: La Playa
```

Funciones sugeridas:

- `clonarEntrega(entrega)`: retorna un objeto entrega nuevo.
- `clonarPedido(pedido)`: retorna un pedido nuevo usando `clonarEntrega`.

Pista: Dibuja cajas para los objetos y flechas para las variables. Después de cada línea que modifica algo, ¿a qué caja apunta cada flecha? Al final del código, ¿cuántos objetos `entrega` existen en memoria?

Ejercicio 11. Pedido de domicilio

Qué practica: Dos objetos que comparten claves: uno con cantidades y otro con precios.

Enunciado:

El programa arranca con el menú de precios:

```
let precios = { arepa: 3500, empanada: 2500, jugo: 4000, tinto: 1500 };
```

El usuario agrega productos y cantidades hasta escribir "fin". Los productos se guardan en un objeto `items` (producto: cantidad). Si pide un producto que ya estaba, la cantidad se suma. Si no está en el menú, se rechaza. Al final se muestra el detalle, el subtotal, el domicilio ($3000 si el subtotal es menor a $20000; si no, gratis) y el total.

Ejemplo de ejecución:

```
[prompt] Producto: empanada
[prompt] Cantidad: 3
[prompt] Producto: jugo
[prompt] Cantidad: 2
[prompt] Producto: empanada
```

```
[prompt] Cantidad: 1
[prompt] Producto: hamburguesa
hamburguesa no está en el menú
[prompt] Producto: fin
Detalle:
empanada x4 = $10000
jugo x2 = $8000
Subtotal: $18000
Domicilio: $3000
Total: $21000
```

Funciones sugeridas:

- `agregarItem(items, producto, cantidad)`: suma o crea la cantidad.
- `calcularSubtotal(items, precios)`: retorna el subtotal.
- `calcularDomicilio(subtotal)`: retorna 3000 o 0.
- `mostrarDetalle(items, precios)`: imprime una línea por producto.

Pista: `items` y `precios` usan las mismas claves. ¿En qué se parece esto a los arrays paralelos de la guía de arrays y qué ventaja tiene? ¿Qué pasa en `agregarItem` la primera vez que llega "empanada"?

Ejercicio 12. ¿Son iguales?

Qué practica: Entender por qué `===` no compara contenido, y comparar dos objetos clave por clave.

Enunciado:

Ejecuta esto y explica el resultado en un comentario:

```
let a = { nombre: "Morral", precio: 85000 };
let b = { nombre: "Morral", precio: 85000 };
console.log(a === b);
```

Después escribe una función que diga si dos objetos tienen exactamente las mismas claves con los mismos valores (solo un nivel, sin objetos adentro). Pruébala con los cuatro casos del ejemplo.

Ejemplo de ejecución:

```
Caso 1, iguales: true
Caso 2, distinto precio: false
Caso 3, a tiene menos claves que b: false
Caso 4, mismas claves en otro orden: true
```

Funciones sugeridas:

- `sonIguales(a, b)`: retorna `true` o `false`.
- `contarClaves(objeto)`: la del ejercicio 5.

Pista: Si solo recorres las claves de `a` y las comparas con `b`, ¿qué caso da un resultado equivocado? ¿Cómo lo arregla comparar primero la cantidad de claves? ¿El orden de las claves debería importar?

Bloque 4. Object.keys, values y entries

Estos tres métodos convierten un objeto en un array. Y con un array ya sabes hacer de todo: `length`, `forEach`, `map`, `filter`, `find`.

```
let inventario = { arepas: 20, empanadas: 8, jugos: 0 };

Object.keys(inventario);     // ["arepas", "empanadas", "jugos"]
Object.values(inventario);   // [20, 8, 0]
Object.entries(inventario);  // [["arepas", 20], ["empanadas", 8], ["jugos", 0]]
```

`Object.entries` retorna un array de arrays: cada elemento es un par donde la posición 0 es la clave y la posición 1 es el valor.

```
Object.entries(inventario).forEach((par) => {
  console.log(par[0] + " tiene " + par[1]);
});
```

Método	Qué retorna	Lo que ya hiciste a mano
`Object.keys`	Array de claves	`for...in` y `contarClaves` (ejercicio 5)
`Object.values`	Array de valores	`for...in` leyendo `objeto[clave]`
`Object.entries`	Array de pares [clave, valor]	`for...in` usando clave y valor a la vez

Ejercicio 13. Boletín 2.0

Qué practica: `Object.keys` y `Object.values` combinados con métodos de array.

Enunciado:

Reescribe el ejercicio 5 sin `for...in`: usa `Object.keys`, `Object.values` y métodos de array. La salida debe ser idéntica. En un comentario responde: ¿qué función del ejercicio 5 desapareció por completo y por qué?

Ejemplo de ejecución:

```
(La misma salida del ejercicio 5)
```

Funciones sugeridas:

- `calcularPromedioObjeto(objeto)`: con `Object.values`.
- `buscarClaveMayor(objeto)`: con `Object.keys`.
- `filtrarClavesMenores(objeto, limite)`: con `Object.keys` y `filter`.

Pista: En `filtrarClavesMenores`, si filtras `Object.keys`, el callback recibe una clave. ¿Cómo llegas a su valor desde adentro del callback?

Ejercicio 14. Reporte de inventario

Qué practica: `Object.entries` con `filter` y `map`.

Enunciado:

Con este inventario:

```
let inventario = { arepas: 20, empanadas: 8, jugos: 0, tintos: 40, pandebonos: 5 };
```

Mostrar cuántos productos hay, el total de unidades, los productos con stock bajo (más de 0 y menos de 10) con su cantidad, y los agotados. Usa `Object.entries`, `Object.values`, `filter` y `map`; nada de `for` ni `for...in`.

Ejemplo de ejecución:

```
Productos: 5
Unidades en total: 73
Stock bajo: empanadas (8), pandebonos (5)
Agotados: jugos
```

Funciones sugeridas:

- `contarUnidades(inventario)`: con `Object.values`.
- `obtenerStockBajo(inventario)`: retorna los pares filtrados.
- `formatearPares(pares)`: retorna un texto como "empanadas (8), pandebonos (5)" usando `map` y `unirConComas`.

Pista: En la prueba de escritorio de `obtenerStockBajo`, ¿qué es `par[0]` y qué es `par[1]` en la segunda vuelta? ¿Qué retorna tu función: claves, valores o pares?

Ejercicio 15. Invertir un diccionario

Qué practica: Construir un objeto nuevo recorriendo `Object.entries`, y manejar valores repetidos.

Enunciado:

Parte A. Con este objeto de códigos de aeropuerto:

```
let codigos = { bogota: "BOG", medellin: "MDE", bucaramanga: "BGA", cali: "CLO" };
```

Construir un objeto invertido (código: ciudad). El usuario escribe códigos hasta "fin" y el programa dice a qué ciudad corresponden.

Parte B. Ahora invierte este objeto de zonas de reparto, donde un valor se repite:

```
let zonas = { centro: "A", laPlaya: "B", pabloVI: "A" };
```

Si lo inviertes como en la parte A, se pierde un barrio. Escribe otra función que agrupe: cada zona debe guardar un array con todos sus barrios.

Ejemplo de ejecución:

```
[prompt] Código: MDE
MDE: medellin
[prompt] Código: SMR
No hay ciudad con el código SMR
[prompt] Código: fin
Zonas agrupadas:
A: centro, pabloVI
B: laPlaya
```

Funciones sugeridas:

- `invertir(objeto)`: retorna el objeto invertido.
- `invertirAgrupando(objeto)`: retorna un objeto cuyos valores son arrays.
- `unirConComas(lista)`: la de la guía de arrays.

Pista: Si usas `invertir` con `zonas`, ¿qué barrio desaparece y por qué? En `invertirAgrupando`, cuando llega una zona que ya existe como clave, ¿qué haces distinto de cuando llega por primera vez? Es el mismo patrón del contador del ejercicio 7.

Antes de entregar

- Cada ejercicio tiene su prueba de escritorio con los datos del ejemplo, incluida la columna del objeto, y coincide con lo que imprime tu programa.
- Ninguna función de cálculo usa `prompt` ni `console.log`.
- En los bloques 1 a 3 no usaste ningún método prohibido.
- El ejercicio 10 tiene tu predicción escrita antes de ejecutar y, si fallaste, la explicación de por qué.
- Probaste al menos un caso borde por ejercicio: una clave que no existe, un texto con mayúsculas, un objeto vacío o un nivel anidado que falta.